

计算机科学笔记示例

Violet

2026-04-07

Chapter 0

目 录

I	第一部分：模板基础与排版	
1.	模板概述	6
1.1.	设计目标	6
1.2.	组件来源说明	6
1.3.	Typst 原生功能演示	6
1.3.1.	脚注 (footnote)	6
1.3.2.	交叉引用 (label + ref)	7
1.3.3.	超链接 (link)	7
1.3.4.	图片 (image)	7
1.3.5.	布局容器	7
2.	标题层级	9
2.1.	二级标题 (Section)	9
2.1.1.	三级标题 (Subsection)	9
3.	文本样式	10
3.0.1.	行内强调	10
3.0.2.	数学公式	10
II	第二部分：列表与表格	
4.	列表样式	12
4.1.	无序列表	12
4.2.	有序列表	12
4.3.	Checklist 复选列表	12
4.4.	树形列表	13
4.5.	圆圈连接线列表	14
5.	表格	15
5.1.	快速表格	15
5.2.	复杂表格	15
III	第三部分：提示框与代码	
6.	提示框	18
6.1.	提示框系统	18

6.2. 提示框实现原理 ····· 18

6.3. 提示框使用建议 ····· 19

7. 代码块 ····· 20

7.1. 行内代码 ····· 20

7.2. 基础代码块 ····· 20

7.2.1. Java 示例 ····· 20

7.2.2. Python 示例 ····· 21

7.2.3. JavaScript 示例 ····· 21

7.2.4. Rust 示例 ····· 21

7.3. 配置与标记语言 ····· 21

7.3.1. JSON 配置 ····· 21

7.3.2. YAML 配置 ····· 22

7.3.3. Dockerfile ····· 22

7.4. 终端命令 ····· 22

8. 算法与流程 ····· 24

8.1. 算法伪代码 ····· 24

8.2. 综合示例 ····· 24

8.2.1. 选择排序的原则 ····· 24

IV

附录

9. 模板速查 ····· 27

9.1. 组件来源速查表 ····· 27

9.2. 可用命令速查 ····· 27

9.3. 提示框类型 ····· 28

9.4. 快捷键说明 ····· 28

10. Typst 原生功能速查 ····· 29

10.1. 文档结构 ····· 29

10.2. 文本格式 ····· 29

10.3. 布局组件 ····· 29

10.4. 数学公式 ····· 29

10.5. 交叉引用 ····· 30

10.6. 外部资源 ····· 30

11. 更新日志 ····· 31

11.1. v1.0.1 (2026-04-06) ····· 31

11.2. v1.0.0 (2026-04-05) ····· 31



第一部分：模板基础与排版

- 1. 模板概述 6
 - 1.1. 设计目标 6
 - 1.2. 组件来源说明 6
 - 1.3. Typst 原生功能演示 6
- 2. 标题层级 9
 - 2.1. 二级标题 (Section) 9
- 3. 文本样式 10

Chapter 1

模板概述

1 设计目标

本模板旨在为计算机科学课程笔记提供一套统一、美观、易用的排版方案。模板集成了常见的排版需求，包括多级标题、代码高亮、提示框、表格和数学公式等，让笔记作者能够专注于内容创作，而非格式调整。

NOTE

本模板基于 Typst 构建，相较于 LaTeX，Typst 具有编译速度快、语法简洁、学习曲线平缓等优势。

模板的核心设计原则：

- **一致性**：所有组件共享统一的配色方案和字体设置
- **可复用性**：组件化设计便于在不同文档中重复使用
- **语义化**：通过颜色和样式区分不同类型的内容

2 组件来源说明

本模板的组件基于以下来源构建：

组件类型	来源	封装方式
表格	<code>\table</code>	<code>\tex-table</code> / <code>\plain-table</code> 封装
无序列表	<code>\list</code>	重写 marker 样式
有序列表	<code>\enum</code>	重写 marker 样式
树形列表	<code>\itemize</code> 包	<code>\tree-list</code> 样式配置
Checklist	<code>\itemize</code> 包	<code>\config.checklist</code> 封装
代码块	<code>\codly</code> 包	<code>\codly-init</code> 全局配置
行内代码	<code>\raw</code>	重写 show 规则
页眉页脚	<code>\page</code>	context 动态内容

3 Typst 原生功能演示

Typst 内置了许多强大的排版功能，本模板已集成以下原生特性：

3.1 脚注 (footnote)

脚注用于添加补充说明或引用来源。模板已定义脚注样式（细线分隔符）。

使用方式：`^[脚注内容]` 语法

这是一个包含脚注的句子^{^[这是一个使用 Typst 原生语法添加的脚注示例。]}，脚注会出现在页面底部。

3.2 交叉引用 (label + ref)

Typst 支持强大的交叉引用功能，可以引用标题、图表、公式等元素。

标签使用 `<name>` 语法直接添加在标题后面，如本节标题 `<sec-cross-ref>`。

引用使用 `@name` 语法：可以引用本节 小节 2。

TIP

标签命名建议：`heading-*` 用于标题，`fig-*` 用于图片，`table-*` 用于表格，`eq-*` 用于公式。

3.3 超链接 (link)

使用 `#link` 创建可点击的超链接：

- 外部链接：[Typst 官网](#)
- 邮箱链接：[发送邮件](#)

3.4 图片 (image)

Typst 原生支持图片插入，使用 `#figure` 函数：

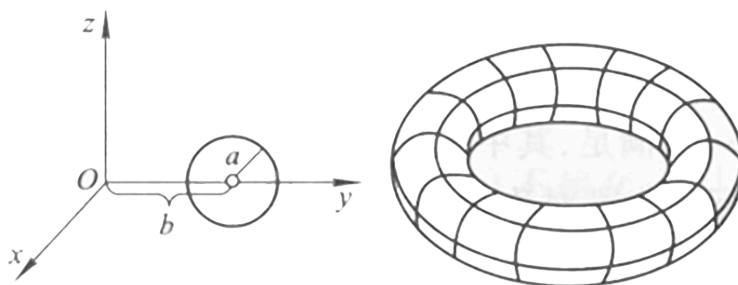


图 1 示例图片（来源：Picsum）

可以使用 `@fig-example` 引用该图片：如图 图 1 所示。

NOTE

`#image` 是 Typst 原生函数，参数依次为：路径/URL、宽度、高度（可选）、caption（可选）。图片会自动编号并加入目录。

3.5 布局容器

Typst 提供多种布局容器：

- `box`：创建带样式的文本容器

- `grid`：网格布局（模板中用于表格）
- `stack`：堆叠布局
- `move`：偏移定位

TIP

这些容器是 Typst 原生组件，本模板中的许多复杂效果（如提示框、标题样式）都是通过组合这些容器实现的。

Chapter 2

标题层级

本模板提供了七级标题体系，每一级都有独特的视觉样式：

1 二级标题 (Section)

二级标题用于划分文档的主要章节，样式为主题色背景的胶囊形状。

1.1 三级标题 (Subsection)

三级标题用于组织章节内的主要内容，样式为蓝色编号前缀。

四级标题 (Sub-subsection)

四级标题用于细分小节，左侧带有蓝色竖线装饰。

五级标题 (Paragraph)

五级标题用于标记独立的知识点，绿色加粗样式便于识别重点。

六级标题 (Subparagraph)

六级标题用于进一步细化内容，绿色常规样式。

TIP

建议实际使用时，最多使用到四级标题。过深的层级结构会增加阅读负担。

Chapter 3

文本样式

0.1 行内强调

模板提供了多种行内强调方式：

- **普通强调**：使用粗体，适合一般强调
- **强调文字**：使用主题色和圆体字体，适合突出重点
- **高亮文字**：带浅色背景，适合标记关键术语
- `src/main/App.java`：文件路径专用样式
- `Ctrl + C`：模拟键盘按键

WARNING

行内强调应适度使用，过多的强调会削弱其效果。建议每个段落不超过 2-3 处强调。

0.2 数学公式

模板支持完整的数学公式排版：

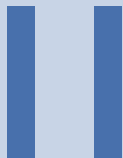
集合框架的时间复杂度分析中，哈希表的查找复杂度接近 $O(1)$ ，而红黑树则为 $O(\log n)$ 。

$$T(n) = T\left(\frac{n}{2}\right) + O(1) \Rightarrow T(n) = O(\log n) < \text{eq-recurrence} > \quad (1)$$

负载因子的定义：

$$\alpha = \frac{n}{m} = \frac{\text{元素数量}}{\text{桶数量}} < \text{eq-load-factor} > \quad (2)$$

可以使用 `@“eq-recurrence”` 和 `@“eq-load-factor”` 引用公式：见公式 `@“eq-recurrence”` 和 `@“eq-load-factor”`



第二部分：列表与表格

- 4. 列表样式 12
 - 4.1. 无序列表 12
 - 4.2. 有序列表 12
 - 4.3. Checklist 复选列表 12
 - 4.4. 树形列表 13
 - 4.5. 圆圈连接线列表 14
- 5. 表格 15
 - 5.1. 快速表格 15
 - 5.2. 复杂表格 15

Chapter 4

列表样式

1 无序列表

一级无序列表用于罗列并列的要点：

- 数据结构是计算机科学的基础
- 算法设计需要考虑时间复杂度和空间复杂度
- 代码实现应当注重可读性和可维护性

二级列表适合展开说明：

- 函数式编程范式
 - 纯函数：不依赖外部状态
 - 不可变性：避免副作用
- 面向对象编程范式
 - 封装：隐藏实现细节
 - 继承：复用代码结构
 - 多态：统一接口多种实现

NOTE

原生无序列表使用 `-` 标记，通过 `set list(marker: ...)` 重写标记样式。本模板定义了 `• ◦ ▪` 三级标记。

2 有序列表

有序列表适合表达步骤或流程：

1. 分析问题需求
2. 设计数据结构和算法
3. 编写代码实现
4. 编写单元测试
5. 优化性能

NOTE

有序列表使用 `+` 标记（Typst 会自动编号）。本模板通过 `show: el.default-enum-list` 增强了样式。

3 Checklist 复选列表

Checklist 适合标记待办事项：

- ☒ 完成模板基础功能

- ☑ 实现代码高亮
- ☐ 添加更多代码语言支持
- ☐ 优化提示框样式
- 📄 完善示例文档
- 🗑️ 移除废弃功能
- ➡️ 向右箭头，表示下一步或前进
- 📅 日历图标，表示日期或时间相关
- ❓ 问号，表示不确定或需要进一步调查
- ❗ 感叹号，表示重要或需要注意
- ★ 星号，表示重点或收藏
- “ ” 引号，表示引用或强调
- 📍 定位图标，表示位置相关
- 🔖 书签图标，表示参考资料或链接
- ℹ️ 信息图标，表示提示或说明
- 💰 钱袋图标，表示成本或预算
- 💡 灯泡图标，表示创意或建议
- 👍 赞图标，表示优点、认可或推荐
- 👎 踩图标，表示缺点、反对或不推荐
- 🔥 火焰图标，表示热门或高优先级
- 🔑 钥匙图标，表示关键点或解决方案
- 🏆 奖杯图标，表示成就或完成
- ⬆️ 向上箭头，表示提升或改进
- ⬇️ 向下箭头，表示降低或放弃

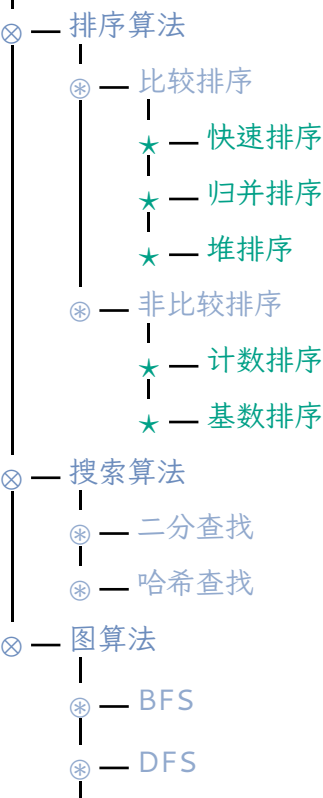
NOTE

Checklist 基于 `itemize` 包的 `config.checklist` 实现。通过 `show: el.config.checklist` 全局启用。

4 树形列表

树形列表适合展示层次结构：

算法分类



* — Dijkstra

NOTE

树形列表基于 `itemize` 包，定义在 `computer-notes.typ` 的 `tree-list` 变量中。树形标记使用数学符号 ($\div$ $\times$ $*$ $\star$)。

5 圆圈连接线列表

1. 分析问题规模，确定算法复杂度要求
 - 数据量大小
 - 实时性要求
2. 选择合适的数据结构
3. 设计算法逻辑
 1. 编写伪代码
 2. 验证边界条件
4. 实现并测试

NOTE

圆圈连接线列表基于 `itemize` 包的 `el.default-enum-list`，配置在 `circle-line-enum` 变量中。使用 `#circle-line-enum[...]` 调用。

Chapter 5

表格

1 快速表格

使用 `#plain-table` 创建普通网格表格（基于原生 `table`，有完整网格线）：

列 1	列 2	列 3
数据 1	数据 2	数据 3
数据 4	数据 5	数据 6
Data7	Data8	Data9
Data10	数据 11	Data12

使用 `#tex-table` 快速创建 tex 风格的表格（基于原生 `table`，仅水平线）：

语言	特点	典型应用
Python	简洁易学	数据分析、AI
Java	跨平台、企业级	后端开发
JavaScript	前端必备	Web 开发
Rust	安全高性能	系统编程

2 复杂表格

复杂功能（如单元格合并）需要直接调用原生 `#table` 函数：

姓名	成绩		总分
	平时	期末	
张三	85	92	177
李四	78	88	166
王五	90	95	185
赵六	82	79	161

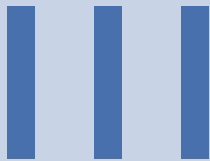
原生 `table` 函数支持：

NOTE

- `table.cell(colspan: n) / table.cell(rowspan: n)`: 单元格合并
- `fill` 参数: 自定义单元格填充色 (函数形式)
- `align` 参数: 单元格对齐方式
- `stroke` 参数: 边框样式

TIP

表格列宽可以使用 `1fr`、`auto` 或固定值 (如 `3cm`) 进行设置。



第三部分：提示框与代码

6. 提示框	18
6.1. 提示框系统	18
6.2. 提示框实现原理	18
6.3. 提示框使用建议	19
7. 代码块	20
7.1. 行内代码	20
7.2. 基础代码块	20
7.3. 配置与标记语言	21
7.4. 终端命令	22
8. 算法与流程	24
8.1. 算法伪代码	24
8.2. 综合示例	24

Chapter 6

提示框

1 提示框系统

模板内置了七种提示框，适用于不同的语义场景：

NOTE

笔记框用于补充说明、背景知识或记忆要点。本模板使用霞鹜文楷作为主要中文字体。添加更多文字以测试多行内容的显示效果和自动换行功能。

TIP

技巧框用于分享实践经验、优化建议或操作捷径。例如，使用 `Alt + Tab` 快速切换窗口。

INFO

信息框用于通用说明或背景知识。测试长内容时的自动换行效果。

WARNING

警告框用于提醒可能出错或需要谨慎处理的情况。例如，删除操作不可恢复。

CAUTION

注意框用于强调容易忽略但影响较大的细节。如版本兼容性、数据迁移风险等。

DANGER

危险框用于标记高风险操作或明确禁止的行为。执行前请务必确认后果。

TODO

待办框用于标记后续需要补充或完善的内容。可配合 Checklist 使用。

2 提示框实现原理

提示框基于以下组件组合实现：

组件	用途
<code>`block`</code>	创建带背景色和边框的容器
<code>`place`</code>	绝对定位标题标签
<code>`rect`</code>	标题背景框
<code>`text`</code>	标题和内容文字

NOTE

`callout` 是提示框的通用构造器函数，定义在 `computer-notes.typ` 中。各个提示框（`note`、`tip` 等）都是对它的调用。

3 提示框使用建议

TIP

- 每种提示框的语义应当保持一致，不要混用
- 避免在单个页面中使用过多提示框
- 提示框内容应简洁明了，一针见血

Chapter 7

代码块

1 行内代码

行内代码像 Markdown 一样是使用成对的反引号（`）包裹，已重写样式：这是一个行内代码示例：

```
"int x = 10;" // 这是注释
```

，适合标记代码片段或命令。

2 基础代码块

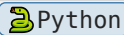
模板使用 Codly 包实现代码语法高亮，支持 30+ 编程语言：

2.1 Java 示例

```
1  import java.util.*;
2
3  public class QuickSort {
4      public static void sort(int[] arr, int low, int high) {
5          if (low < high) {
6              int pi = partition(arr, low, high);
7              sort(arr, low, pi - 1);
8              sort(arr, pi + 1, high);
9          }
10     }
11
12     private static int partition(int[] arr, int low, int high) {
13         int pivot = arr[high];
14         int i = low - 1;
15         for (int j = low; j < high; j++) {
16             if (arr[j] < pivot) {
17                 i++;
18                 swap(arr, i, j);
19             }
20         }
21         swap(arr, i + 1, high);
22         return i + 1;
23     }
24 }
```

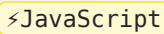
2.2 Python 示例

```
1 def quicksort(arr):  
2     if len(arr) <= 1:  
3         return arr  
4     pivot = arr[len(arr) // 2]  
5     left = [x for x in arr if x < pivot]  
6     middle = [x for x in arr if x == pivot]  
7     right = [x for x in arr if x > pivot]  
8     return quicksort(left) + middle + quicksort(right)
```



2.3 JavaScript 示例

```
1 const quicksort = (arr) => {  
2     if (arr.length <= 1) return arr;  
3     const pivot = arr[Math.floor(arr.length / 2)];  
4     const left = arr.filter(x => x < pivot);  
5     const middle = arr.filter(x => x === pivot);  
6     const right = arr.filter(x => x > pivot);  
7     return [...quicksort(left), ...middle, ...quicksort(right)];  
8 };
```



2.4 Rust 示例

```
1 fn quicksort<T: Ord + Clone>(arr: &[T]) -> Vec<T> {  
2     if arr.len() <= 1 {  
3         return arr.to_vec();  
4     }  
5     let pivot = arr[arr.len() / 2].clone();  
6     let left: Vec<T> = arr.iter().filter(|x| **x < pivot).cloned().collect();  
7     let middle: Vec<T> = arr.iter().filter(|x| **x == pivot).cloned().collect();  
8     let right: Vec<T> = arr.iter().filter(|x| **x > pivot).cloned().collect();  
9  
10    let mut result = quicksort(&left);  
11    result.extend(middle);  
12    result.extend(quicksort(&right));  
13    result  
14 }
```



3 配置与标记语言

3.1 JSON 配置

```
1 {  
2     "name": "computer-notes",
```



```
3   "version": "1.0.0",
4   "author": "Violet",
5   "features": {
6     "syntax-highlight": true,
7     "auto-toc": true,
8     "multiple-themes": false
9   }
10 }
```

3.2 YAML 配置

```
1 database:
2   host: localhost
3   port: 5432
4   name: myapp
5
6 logging:
7   level: info
8   format: json
```

YAML

3.3 Dockerfile

```
1 FROM node:18-alpine AS builder
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci --only=production
5 COPY . .
6 RUN npm run build
7
8 FROM nginx:alpine
9 COPY --from=builder /app/dist /usr/share/nginx/html
10 EXPOSE 80
11 CMD ["nginx", "-g", "daemon off;"]
```

Docker

4 终端命令

终端样式适合展示命令行操作：

```
git status
```

```
git commit -m "feat: add quicksort implementation"
```

```
docker compose up -d --build
```

NOTE

终端样式是一个自定义组件（`terminal` 函数），基于 `block` 和 `text` 构建，模拟深色终端窗口外观。

Chapter 8

算法与流程

1 算法伪代码

Algorithm

快速排序算法：

- 输入：数组 arr，左边界 low，右边界 high
- 如果 low < high：
 - 选取基准元素 pivot = arr[high]
 - 划分：小于 pivot 的放左边，大于的放右边
 - 递归排序左右两部分
- 输出：已排序的数组

NOTE

`algorithm` 函数基于 `block` 组件，添加渐变背景和标题栏样式。

2 综合示例

下面用一个小主题串起多个组件：

2.1 选择排序的原则

在选择集合类型时，需要综合考虑以下因素：

场景	推荐数据结构	原因
快速查找	HashMap	O(1) 时间复杂度
有序遍历	TreeMap	自动维护顺序
频繁插入删除	LinkedList	O(1) 操作
去重计数	HashSet	自动去重

WARNING

不要过早优化。在没有性能问题的情况下，应该优先选择实现简单、易于维护的数据结构。

具体选择时：

- 如果强调随机访问，优先考虑 **ArrayList** 或 **HashMap**
- 如果强调元素顺序，优先考虑 **TreeMap** 或 **LinkedList**

- 如果强调并发安全，优先考虑 **ConcurrentHashMap**

核心原则：根据实际需求选择最合适的数据结构，而非追求“最优”解决方案。

IV

附录

9. 模板速查	27
9.1. 组件来源速查表	27
9.2. 可用命令速查	27
9.3. 提示框类型	28
9.4. 快捷键说明	28
10. Typst 原生功能速查	29
10.1. 文档结构	29
10.2. 文本格式	29
10.3. 布局组件	29
10.4. 数学公式	29
10.5. 交叉引用	30
10.6. 外部资源	30
11. 更新日志	31
11.1. v1.0.1 (2026-04-06)	31
11.2. v1.0.0 (2026-04-05)	31

Chapter 9

模板速查

1 组件来源速查表

功能	底层来源	模板封装
标题 1-6 级	<code>`heading`</code>	<code>`show heading.where(level: n)`</code>
列表	<code>`list`</code> / <code>`enum`</code>	<code>`set list(marker: ...)`</code>
树形列表	<code>`itemize`</code> 包	<code>`show: tree-list`</code>
Checklist	<code>`itemize`</code> 包	<code>`show: el.config.checklist`</code>
表格	<code>`table`</code>	<code>`tex-table`</code> / <code>`plain-table`</code>
代码块	<code>`codly`</code> 包	<code>`codly-init`</code> / <code>`codly`</code>
行内代码	<code>`raw`</code>	<code>`show raw`</code>
提示框	<code>`block`</code> + <code>`place`</code>	<code>`note`</code> / <code>`tip`</code> 等函数
页眉页脚	<code>`page`</code>	<code>`set page(header/footer: ...)`</code>
数学公式	<code>`\$`</code> / <code>`math`</code>	原生支持
脚注	<code>`footnote`</code>	原生支持 + 样式定制
交叉引用	<code>`label`</code> + <code>`ref`</code>	原生支持
链接	<code>`link`</code>	原生支持 + 样式定制

2 可用命令速查

命令	用途	示例
<code>#file("path")</code>	文件路径	<code>#file("src/main/App.java")</code>
<code>#highlight("text")</code>	高亮文字	<code>#highlight("重点")</code>
<code>#emphasis("text")</code>	强调文字	<code>#emphasis("关键点")</code>
<code>#shortcut("keys")</code>	快捷键	<code>#shortcut("Ctrl + S")</code>
<code>#terminal("cmd")</code>	终端命令	<code>#terminal("npm install")</code>
<code>#algorithm[内容]</code>	算法伪代码	<code>#algorithm[...]</code>

(`#fancy-divider` , "装饰分隔线", `#fancy-divider`),)

3 提示框类型

NOTE

#note[内容]: 笔记

TIP

#tip[内容]: 技巧

INFO

#info[内容]: 信息

WARNING

#warning[内容]: 警告

CAUTION

#caution[内容]: 注意

DANGER

#danger[内容]: 危险

TODO

#todo[内容]: 待办

4 快捷键说明

- 编译文档: `Ctrl + Shift + B`
- 预览文档: `Ctrl + Shift + P`
- 格式化代码: `Shift + Alt + F`

Chapter 10

Typst 原生功能速查

1 文档结构

功能	语法	说明
文档元信息	<code>#set document(...)</code>	标题、作者、日期
页面设置	<code>#set page(...)</code>	纸张、边距、页眉页脚
目录	<code>#outline(depth: n)</code>	自动生成目录

2 文本格式

功能	语法	说明
加粗	<code>*文本*</code>	粗体显示
斜体	<code>_文本_</code>	斜体显示
等宽	<code>`代码`</code>	代码字体
上标	<code>^^{文本}</code>	上标文字，配对使用 <code>^...^</code>
下标	<code>~~{文本}</code>	下标文字，配对使用 <code>~...~</code>

3 布局组件

组件	语法	用途
块	<code>#block[...]</code>	独占一行
盒	<code>#box[...]</code>	行内容器
网格	<code>#grid(...)</code>	二维布局
堆叠	<code>#stack(...)</code>	垂直/水平堆叠
对齐	<code>#align(...)</code>	对齐内容
定位	<code>#place(...)</code>	绝对/相对定位

4 数学公式

数学公式使用 `$...$` 包裹，支持复杂的数学排版：

- 行内公式: `$x^2 + y^2 = z^2$`
- 独立公式: `$$...$$` 或 `$ ... $ + #set math.equation(numbering: ...)`
- 常用符号: `sum`, `prod`, `int`, `infty`, `alpha`, `beta` 等
- 矩阵: `matrix(...)` 函数
- 分式: `1/2` 或 `\(a)/(b)`

5 交叉引用

功能	语法	示例
定义标签	<code><name></code>	在标题/公式末尾添加，如 <code>`== 标题 <sec-intro>`</code>
引用	<code>@name</code>	<code>`@sec-intro`</code> 引用标题/公式/图片
公式标签	<code><"eq-name"></code>	数学公式中用引号包裹，如 <code>`<"eq-abc">`</code>

6 外部资源

资源	语法	说明
图片	<code>#figure(image(...), caption: [...])</code>	带编号和标题
图片	<code>#image("path", width: 60%)</code>	纯图片，无编号
链接	<code>#link("url")[text]</code>	可点击超链接
脚注	<code>^[脚注内容]</code>	快捷语法（推荐）
脚注	<code>#footnote([内容])</code>	函数形式

Chapter 11

更新日志

1 v1.0.1 (2026-04-06)

- 添加详细的组件来源说明
- 新增 Typst 原生功能演示（脚注、交叉引用、链接等）
- 添加完整的模板速查表
- 修正脚注语法速查表（`^[...]` 为推荐语法）
- 修正交叉引用语法说明（标签直接放在标题/公式后）
- 修正文本格式速查表（上标 `^{\dots}`、下标 `_{\dots}`）
- 修正 circle-line-enum 来源说明（基于 `el.default-enum-list`）

2 v1.0.0 (2026-04-05)

- 初始版本发布
 - 实现七级标题体系
 - 集成 Codly 代码高亮
 - 添加七种提示框
 - 支持表格
 - 实现树形列表和圆圈连接线列表
-

本文档完